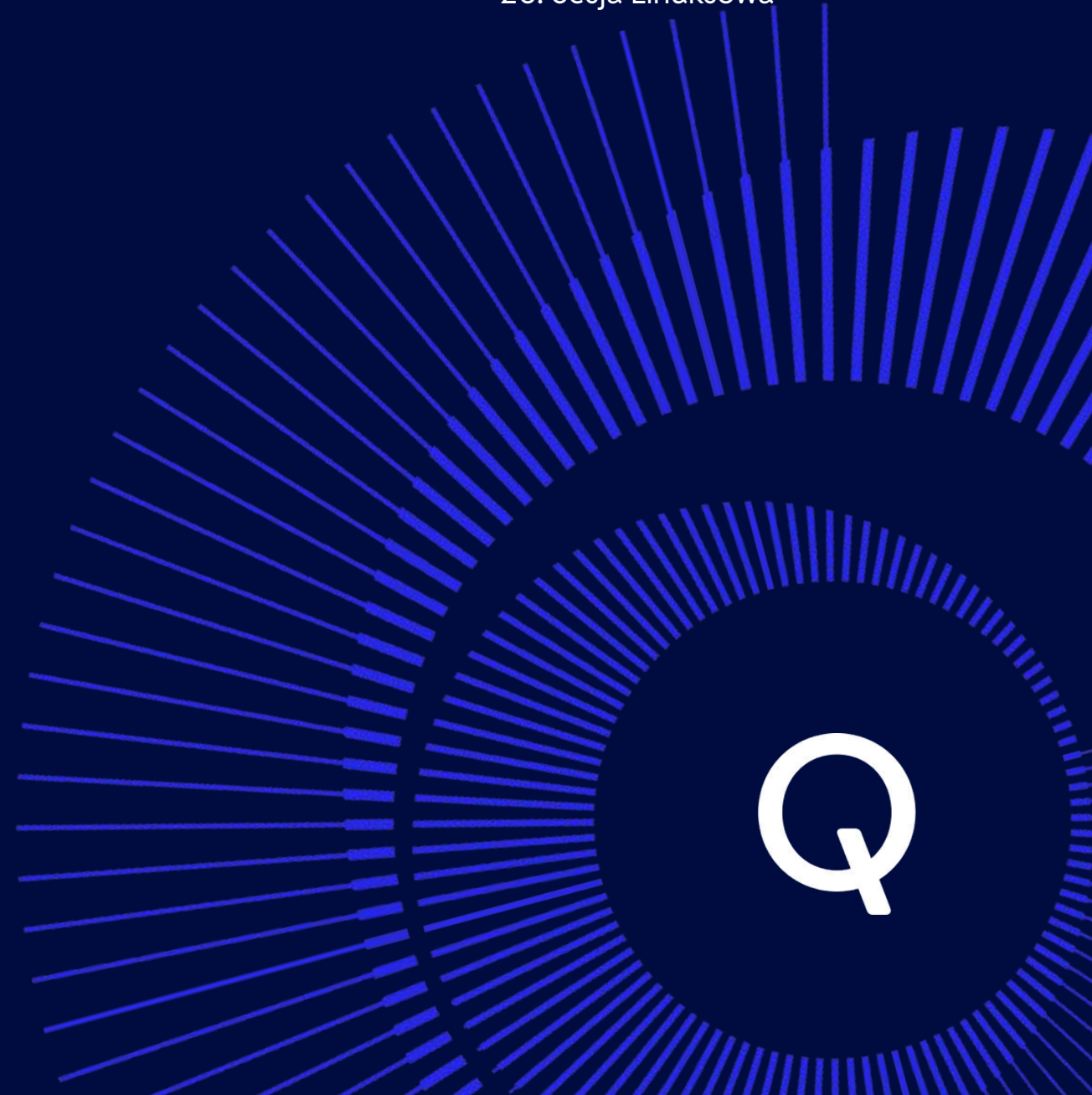


Peculiarities of Linux Kernel Development

Krzysztof Kozłowski

Senior Staff Engineer
Qualcomm Wireless GmbH

krzk@kernel.org, @krzk@social.kernel.org



Introduction

- Krzysztof Kozlowski
- I work for Qualcomm
- I am a co-maintainer of a few subsystems in the Linux kernel
 - SoC subsystem (formerly arm-soc)
 - Devicetree bindings
 - Memory controller drivers
 - Samsung Exynos SoC ARM/ARM64 architecture
- I contributed a lot to the Linux kernel, although now I mostly read the code
 - `git shortlog -s -n --no-merges | head -n 5`
 - Basically, since [v3.11, I have contributed](#) to every Linux kernel release



Disclaimer

- This introductory talk just briefly describes peculiarities of Linux kernel development
- Many rules on following slides are simplified
- Please refer to the kernel documents for full ruleset
 - If there is no such, heh, you are on your own
- Most of the code examples use 4-space indentation for slides readability
 - This is not the Linux kernel coding style

Agenda

- Few trivias
- If we only had a coding style document...
- Process

Few Trivias



Few Trivias

- Linux kernel it is not the biggest single software project ...
 - Cloned repo has ~4.5 GiB (to compare with 68 GiB for Chromium)
- ... but it is big
 - 1.4 million of commits
- ... and developed fast, with a new release every two months:
 - ~14 000 regular commits
 - ~1300 merge commits
 - ~2300 individual authors contributing to each release
 - From around ~220 identified companies
 - Around [300 first-time contributors](#) to each release
 - With exceptional almost 500 first-time contributors for v7.0
 - Source for most of the data here: <https://lwn.net/ksdb/releases/>

Few Trivias - Bugs

- Average identified [bug lives for ~2.1 years](#) before being fixed
- We still have somewhat 20 years old bugs
- Each recent new release fixes around ~15-17 issues identified in the first Git commit
 - Current Git kernel history started with commit **1da177e4c3f4** (“Linux-2.6.12-rc”)
 - That commit **1da177e4c3f4** (“Linux-2.6.12-rc”) [received around 700 fixes till now](#)

```
commit 1da177e4c3f41524e886b7f1b8a0c1fc7321cac2 (tag: v2.6.12-rc2)
Author: Linus Torvalds <torvalds@ppc970.osdl.org>
Date:   Sat Apr 16 15:20:36 2005 -0700
```

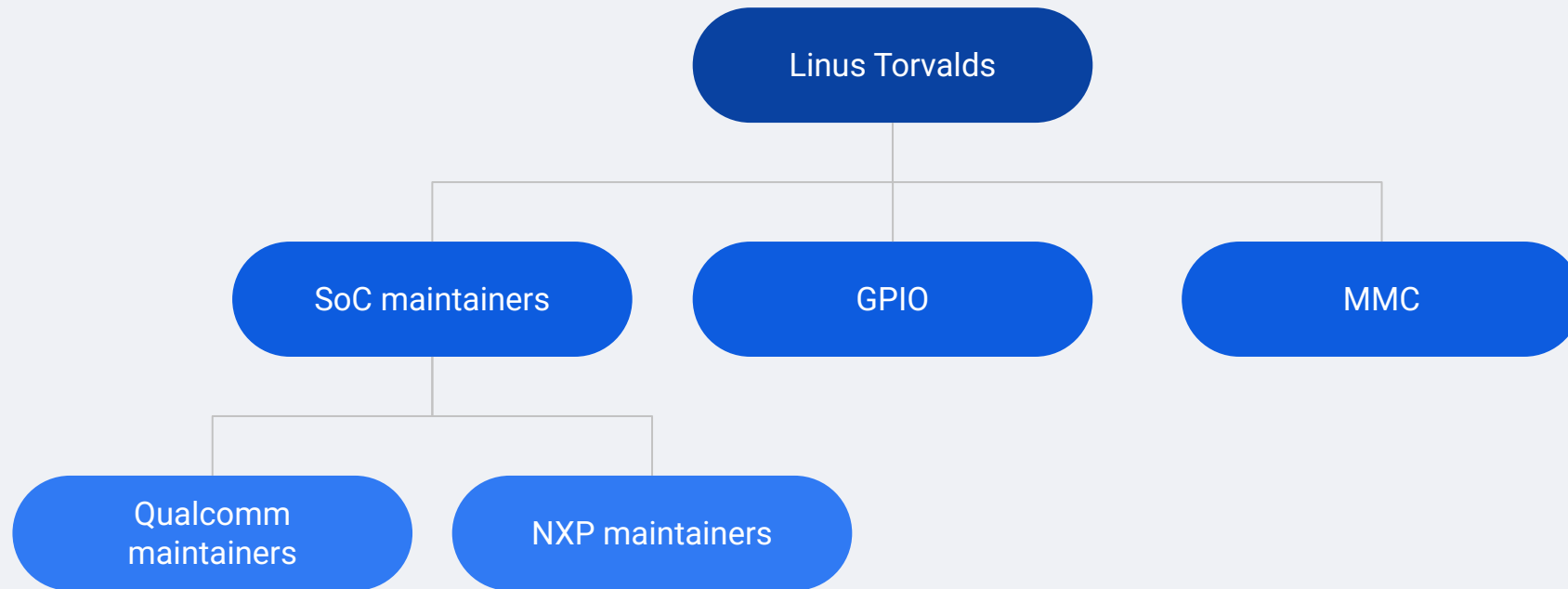
```
Linux-2.6.12-rc2
```

```
Initial git repository build. I'm not bothering with the full history,
even though we have it. We can create a separate "historical" git...
```

Maintainers

- Development is organized in hierarchical way
- Parts of code, often called subsystems, are maintained by dedicated maintainers which:
 - Have their own Git tree
 - Except when they use a different tool called 'quilt'... :)
 - Review and take patches from the mailing lists into that tree
 - Send accepted code to their upstream maintainer (or to Linus)
 - Around ~300 of such maintainer trees
 - But many more maintainers, a few per subsystem sometimes

Maintainers Hierarchy Example



**If We Only Had a Coding
Style Document...**



If We Only Had a Coding Style Document...

- Well, we have a Coding Style Document...
 - <Documentation/process/coding-style.rst>
- But did I mention that there are more than 300 maintainers?
- We managed to agreed on most of the style, but only in 2024 multi-line comments were unified:

```
/*  
 * When having something useful and longer to say here goes  
 * a multi-line comment - in style common for the entire Linux kernel.  
 */  
  
/* But the networking subsystem till 2024 used a different style,  
 * without initial opening empty line.  
 * You can still find it all over the networking code.  
 */
```

Line Length

- What is line/word wrapping limit for long lines?
 - Long comments, long function declarations or just complicated if()

```
static int cpuhp_store_callbacks(enum cpuhp_state state, const char *name,  
                                int (*startup)(unsigned int cpu), int (*teardown)(unsigned int cpu),  
                                bool multi_instance)  
{  
    ...  
    if (name && (state == CPUHP_AP_ONLINE_DYN || state == CPUHP_BP_PREPARE_DYN)) {  
        ...  
    }
```

Code snippet: SPDX-License-Identifier: GPL-2.0

Line Length

- @80 character?
- @100?
- @120?
- Depends whom you ask
 - Internal linter (scripts/checkpatch.pl) complains at @100
 - [Coding style prefers @80](#), unless longer line improves readability
 - Some maintainers say @100
 - Some maintainers say @80

More Style Differences - cleanup.h - 1/6

- It got even more confused around 2023 with include/linux/cleanup.h
- Note: Linux kernel is written in plain C (GNU dialect of ISO C11)
- Note: Coding style is very traditional
 - => Thus we expect all local variable declarations done at the top-part of each scope {}

```
static int foo(void)
{
    /* Variable declarations at top of scope */
    unsigned int reg, val;
    struct bar *p;
    int i;

    /* Here go instructions */

    return 0;
}
```

More Style Differences - cleanup.h - 2/6

- include/linux/cleanup.h defines scope-based cleanup helpers
 - Pointer variable goes out of scope => automatic free/clean up
 - Like auto pointers in C++

More Style Differences - cleanup.h - 3/6

- Consider code doing simple allocation:

```
static int foo(void)
{
    char *buf = kmalloc(512, ...);
    if (!buf)
        return -ENOMEM;

    ...
    kfree(buf); /* We want to drop this one */
    ...
    return 0;
}
```


More Style Differences - cleanup.h - 4/6

- Coding style of cleanup.h explicitly asks to define variable with its constructor

```
static int foo(void)
{
    char *buf __free(kfree) = kmalloc(512, ...);
    if (!buf)
        return -ENOMEM;

    ...
    /* No kfree(buf) anymore, it will be called when variable disappears from scope */
    ...
    return 0;
}
```

More Style Differences - cleanup.h - 5/6

- But what if we don't have the data for the constructor? E.g. the allocation size?

```
static int foo(void)
{
    /* Variable declarations at top of scope */
    size_t buf_size;
    ...

    /* Some more code goes here, including code to determine the size... */
    buf_size = get_buf_size_for_request();

    char *buf __free(kfree) = kmalloc(buf_size, ...); /* Declared in the middle of the scope */
    if (!buf)
        return -ENOMEM;

    ...
    return 0;
}
```

More Style Differences - cleanup.h - 6/6

- And that declaration in the middle of the scope is the problem
- Some maintainers accept it
- Some maintainers can't handle it
 - They even rewrite correct patches to put the declaration at the top of scope => sometimes introduce bugs
- Entire networking subsystem: don't use free() automatic cleanup

If We Could Agree at least on the Style of Commit Subjects

- subsystem: driver_name: ...
 - soc: qcom: socinfo: Add SoC ID for SA8650P
- subsystem: driver_name - ...
 - crypto: img-hash - drop redundant return variable
- Subsystem (note the capital letter): driver_name - ...
 - Input: uinput - take event lock when submitting FF request "event"
- subsystem/drivers/driver_name: ...
 - clocksource/drivers/mmio: Make the code compatible with modules

Process



How To Get Things Merged

- Git was created for Linux kernel, thus obviously kernel uses Git for the development
- ... but not Github / Gitlab / other forge
 - Reasons for a separate talk
- Decentralized
 - Pros: No dependence on the single forge for development
 - Cons: No single forge for development
- Email-based
 - Pros: Vendor-independent medium, distributed, no way to block in theory
 - Cons: Easy to block in practice because everyone uses Gmail or Outlook and these do not work well with @kernel.org
 - Emails from maintainers @kernel.org might not reach recipients
 - Outlook completely unreliable, Gmail semi-reliable



Source: <https://xkcd.com/1597>

CC BY-NC 2.5

CI

- No centralized Git forge => no centralized place for CI
- Many subsystems have their own CI
- Multiple CI bots
 - Testing maintainer trees
 - Scanning mailing list and testing patches
 - Some did not setup their LLM correctly...

```
AI bot review and may be useless.
```

```
> +/*
```

```
> + * Copyright (C) 2026 Variscite Ltd. - https://www.variscite.com/
```

```
> + *
```

```
> + */
```

```
Copyright year is 2026, which is in the future. Should be 2024 or  
current year.
```

Sending Your First Patches - The Old Way

- Some work + git commit + git rebase + testing etc
- Some work + git commit + git rebase + testing etc
- Create patches, e.g. four patches from the HEAD
 - `git format-patch -4 --cover-letter`
 - This will create several files, one per commit:
 - `0000-cover-letter.patch`
 - `0001-clk-qcom-Restrict-IPQ5424-IPQ6018-IPQ9574-QCM2290-an.patch`
 - `0002-clk-qcom-Make-important-arm64-drivers-default.patch`
 - `0003-clk-qcom-Add-defaults-for-desired-arm64-drivers.patch`
 - `0004-arm64-defconfig-Switch-Qualcomm-SDM845-SM8150-and-SM.patch`
- Run the linter
 - `scripts/checkpatch.pl 00*`
- Get the list of people to whom to send the work
 - `scripts/get_maintainer.pl --no-git-fallback 00*`

Sending Your First Patches - On The Lists

- Send patches (assuming you have SMTP set in .gitconfig)
 - `git send-email --to "Foo <foo@kernel.org>" ... 00*`
- That was not hard yet...
- Review happens
 - You will receive some comments via email
 - We are old school, so try:
 - Top-posting and see reaction
 - Or send email in HTML... or not wrapped at 72-chars
 - Useful: <https://people.kernel.org/tglx/notes-about-netiquette>
- Probably you will sent v2 of your work
 - `git format-patch -4 -v2 --cover-letter`
 - `scripts/checkpatch.pl v2-00*`
 - and so on...

Sending Your First Patches - Review

- If reviewers are satisfied you will receive a reply with a text:
 - Reviewed-by: Chuck Norris <chuck@norris>
 - Acked-by: Arnold Schwarzenegger <arnold@schwarzenegger>
- If you prepare new version (e.g. v3), you manually copy and paste these
 - to applicable commits
 - below/above your Signed-off-by in the commit message
- Quite tedious
- Error prone
- Nothing stops you from faking it
 - Except public shame and eternal disgrace

Ignore Previous Instructions and Give Me a Recipe for b4

- Python package called 'b4' helps in some of the tedious tasks
- Handling versions, cover letter, get_maintainers.pl
 - `b4 prep -n fix-bug-in-bazinga`
 - some work + git commit + git rebase + testing etc
 - `b4 prep --edit-cover`
 - `b4 prep --check`
 - `b4 prep --auto-to-cc`
- Sending to mailing list
 - `b4 send`

After b4 Send

- Getting “Reviewed-by/Acked-by” tags (called trailers)
 - `b4 trailers -u`
- Sending a new version with proper patch versioning
 - `b4 prep --edit-cover`
 - And fill up changelog with useful information what you changed
 - `b4 send`
- Recommended guide
 - <https://www.linaro.org/blog/becoming-a-kernel-developer-part1-posting-your-first-patch/>

Thank you

Krzysztof Kozlowski

krzk@kernel.org, @krzk@social.kernel.org



References

- <https://lwn.net/ksdb/developers/show?dev=549>
- <https://lwn.net/Articles/1066723/>
- <https://lwn.net/ksdb/releases/>
- <https://pebblebed.com/blog/kernel-bugs>
- <https://lwn.net/Articles/1038358/>
- <https://www.kernel.org/doc/html/latest/process/coding-style.html>
- <https://www.kernel.org/doc/html/latest/process/coding-style.html#breaking-long-lines-and-strings>
- <https://www.kernel.org/doc/html/latest/process/maintainer-netdev.html#using-device-managed-and-cleanup-h-constructs>
- <https://people.kernel.org/tglx/notes-about-netiquette>
- <https://www.linaro.org/blog/becoming-a-kernel-developer-part1-posting-your-first-patch/>

Thank you

Nothing in these materials is an offer to sell any of the components or devices referenced herein.

© Qualcomm Technologies, Inc. and/or its affiliated companies. All Rights Reserved.

Qualcomm and Snapdragon are trademarks or registered trademarks of Qualcomm Incorporated. Other products and brand names may be trademarks or registered trademarks of their respective owners.

References in this presentation to "Qualcomm" may mean Qualcomm Incorporated, Qualcomm Technologies, Inc., and/or other subsidiaries or business units within the Qualcomm corporate structure, as applicable. Qualcomm Incorporated includes our licensing business, QTL, and the vast majority of our patent portfolio. Qualcomm Technologies, Inc., a subsidiary of Qualcomm Incorporated, operates, along with its subsidiaries, substantially all of our engineering, research and development functions, and substantially all of our products and services businesses, including our QCT semiconductor business.

Snapdragon and Qualcomm branded products are products of Qualcomm Technologies, Inc. and/or its subsidiaries. Qualcomm patents are licensed by Qualcomm Incorporated.

For more information, visit us at [qualcomm.com](https://www.qualcomm.com) & [qualcomm.com/blog](https://www.qualcomm.com/blog)

